

QuickCourse

Sistem Microservices Pendaftaran Mata Kuliah

Dokumen ini adalah spesifikasi tugas akhir Pelatihan Backend Golang + Microservices. Tugas ini tidak menguji seberapa banyak kode yang bisa kamu tulis dalam satu hari, melainkan seberapa dalam kamu memahami keputusan-keputusan yang sudah dilatih selama tiga minggu terakhir — dan apakah kamu bisa mempertanggungjawabkannya.

Program	Pelatihan Backend Golang + Microservices — 4 Minggu
Pelaksanaan	Pertemuan Minggu 4 (8 jam efektif), dilanjutkan demo di sesi sore
Peserta	Mahasiswa — individu atau kelompok
Prasyarat	Target Minggu 1, 2, dan 3 sudah tercapai
Teknologi	Go, Gin, PostgreSQL, Docker & Docker Compose, Git
Bentuk Penilaian	Demo <i>live</i> end-to-end + sesi tanya-jawab pertahanan
Bobot	Penentu akhir untuk aspek Implementasi Teknis (40%) dan Presentasi Akhir (15%)

Aturan tunggal yang paling menentukan nilai: kode yang berjalan adalah syarat minimum, bukan nilai. Yang dinilai adalah kemampuan menjelaskan *mengapa* kode itu ditulis seperti itu. Bagian yang tidak bisa kamu jelaskan saat ditanya akan dianggap tidak dikerjakan.

Dokumen spesifikasi tugas akhir • dibagikan pada akhir pertemuan Minggu 3

1. Tentang Tugas Akhir Ini

Apa yang sebenarnya sedang diuji.

Selama tiga minggu kamu belajar tiga hal yang terpisah: menulis Go yang idiomatis, membangun REST API berlapis di atas PostgreSQL, dan memecah sistem menjadi beberapa service yang saling memanggil. Tugas akhir ini menggabungkan ketiganya menjadi satu sistem yang harus benar-benar berjalan — lengkap dengan bagian yang paling sering diabaikan: *apa yang terjadi ketika sesuatu gagal*.

Studi kasus ini sengaja dirancang supaya tidak bisa diselesaikan hanya dengan menyalin kode CRUD. Ada aturan bisnis yang menuntut kamu memahami transaksi database, ada pemanggilan antar-service yang menuntut kamu memahami `context` dan timeout, dan ada pertanyaan pertahanan yang menuntut kamu memahami alasan di balik setiap struktur yang kamu pakai.

Peta materi yang diuji

Materi	Diuji lewat	Kalau tidak paham, gejalanya
MINGGU 1 Struct, interface, error handling, testing	Bagian 3 (spesifikasi teknis), Bagian 6 (kontrak error), unit test wajib	Repository dipanggil langsung sebagai struct, error ditelan atau di- <code>panic</code> , test tidak bisa ditulis karena semua saling terikat
MINGGU 2 REST API, layered architecture, PostgreSQL	Bagian 4 (endpoint), Bagian 5 (aturan bisnis), Bagian 6 (format response)	Logika bisnis menumpuk di handler, query SQL tersebar di mana-mana, format response beda-beda tiap endpoint
MINGGU 3 Microservices, komunikasi REST, timeout, Docker Compose	Bagian 2 (kepemilikan data), Bagian 5 (BR-05, BR-06), Bagian 8 (uji penerimaan)	Satu service menulis langsung ke database service lain, aplikasi menggantung saat service tetangga mati, sistem hanya jalan di laptop sendiri

Cara membaca dokumen ini

Bagian 3–6 adalah kontrak: semuanya wajib. Bagian 7 menentukan sampai tingkat mana kamu menyelesaikannya. Bagian 8 adalah skenario yang akan dijalankan di depanmu saat demo — jadikan itu checklist harianmu. Bagian 11 adalah daftar pertanyaan yang akan ditanyakan; tidak ada pertanyaan kejutan di luar daftar itu.

2. Deskripsi Sistem QuickCourse

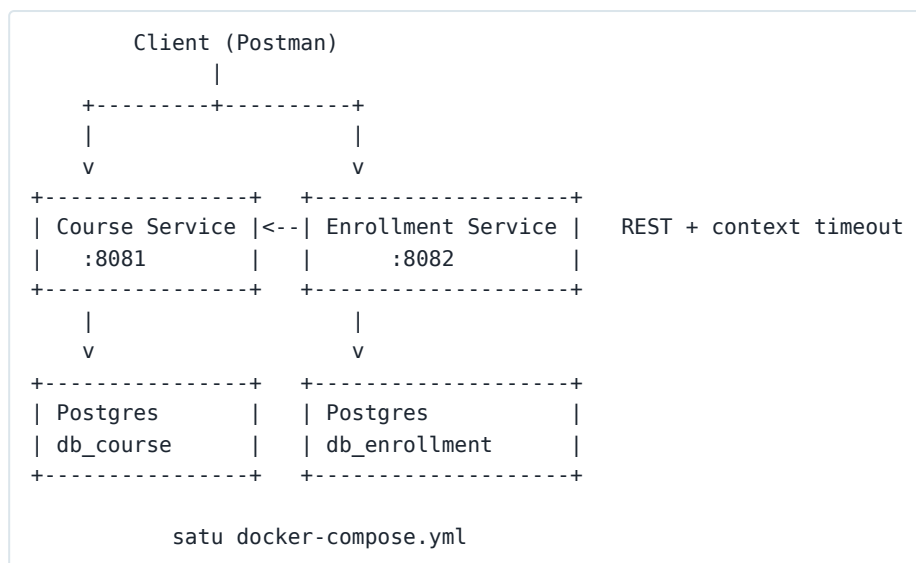
Konteks masalah dan batas tanggung jawab tiap service.

Situasi

Sebuah kampus membuka pendaftaran kelas pilihan secara online. Setiap kelas punya kuota terbatas dan ratusan mahasiswa membuka halaman pendaftaran di menit yang sama begitu sistem dibuka. Sistem lama berbentuk satu aplikasi utuh: ketika bagian pendaftaran ramai, seluruh sistem ikut lambat, dan tim akademik tidak bisa mengubah data mata kuliah tanpa menyentuh kode pendaftaran.

Kampus memutuskan memecah sistem menjadi dua service yang berdiri sendiri, masing-masing dengan databasenya sendiri. Kamu bertugas membangunnya.

Arsitektur target



Pembagian tanggung jawab

Service	Tanggung jawab	Data yang dimiliki
Course Service port 8081	Mengelola master data mata kuliah: kode, nama, deskripsi, dosen pengampu, kapasitas. Satu-satunya pihak yang boleh mengubah jumlah kursi terpakai.	<code>courses</code>
Enrollment Service port 8082	Mengelola pendaftaran mahasiswa ke suatu course. Tidak tahu apa-apa tentang isi tabel course — ia hanya bisa bertanya dan meminta lewat HTTP.	<code>enrollments</code>

Aturan yang tidak bisa ditawar: kepemilikan data

Enrollment Service **dilarang** membuka koneksi ke database Course Service, dalam bentuk apa pun — termasuk "cuma untuk SELECT". Kalau Enrollment butuh tahu sisa kuota, ia harus memanggil HTTP endpoint Course Service. Melanggar aturan ini membuat seluruh aspek Minggu 3 dinilai nol, walaupun sistemnya berjalan mulus.

3. Ruang Lingkup Teknis Wajib

Ketentuan yang berlaku untuk kedua service.

Aspek	Ketentuan	Materi
Struktur proyek	Setiap service adalah Go Module tersendiri dengan package yang terpisah menurut peran. <code>main.go</code> hanya boleh berisi perakitan dependency dan menjalankan server — tanpa logika bisnis dan tanpa query SQL.	M1
Arsitektur berlapis	Handler → Service → Repository. Ketergantungan disuntikkan lewat constructor (<code>NewEnrollmentService(repo, client)</code>), bukan variabel global dan bukan <code>init()</code> .	M2

Interface	Repository dideklarasikan sebagai interface di sisi yang memakainya (layer service), bukan di sisi yang mengimplementasikannya. Service tidak boleh menyebut tipe konkret repository.	M1
Error handling	Setiap service mendefinisikan <i>sentinel error</i> untuk kondisi domainnya (<code>ErrCourseNotFound</code> , <code>ErrNoSeatAvailable</code> , <code>ErrAlreadyEnrolled</code> , ...). Error dari lapisan bawah dibungkus dengan <code>fmt.Errorf("...: %w", err)</code> dan diperiksa di lapisan atas dengan <code>errors.Is . panic</code> dilarang dipakai sebagai alur kontrol.	M1
Unit test	Minimal 3 test table-driven di layer service pada Enrollment Service, dengan repository dan HTTP client digantikan implementasi tiruan (<i>fake/mock</i>). Wajib mencakup satu kasus sukses, satu kasus kuota habis, dan satu kasus Course Service gagal merespons. <code>go test ./...</code> harus lulus.	M1
HTTP layer	Gin. Semua request body divalidasi lewat struct tag <code>binding</code> . Path/query parameter divalidasi sebelum masuk ke service.	M2
Middleware	Minimal <i>request logging</i> (method, path, status, durasi) dan <i>recovery</i> . Log harus terstruktur dan menyertakan nama service, supaya log dua service bisa dibedakan di satu terminal.	M2
Database	PostgreSQL, satu database per service. Akses memakai <code>database/sql</code> dengan driver <code>pgx</code> (tanpa ORM). Skema dibuat lewat file <code>init.sql</code> yang otomatis dijalankan container Postgres.	M2
Konfigurasi	Host, port, dan kredensial dibaca dari environment variable. Dilarang menuliskan <i>connection string</i> atau URL service lain secara <i>hardcode</i> di dalam kode.	M2
Komunikasi antar service	REST. Wajib memakai <code>http.Client</code> dengan <code>context.WithTimeout</code> maksimal 3 detik per pemanggilan. Response non-2xx dari service lain wajib diterjemahkan menjadi error domain, bukan diteruskan mentah-mentah.	M3
Containerization	<code>Dockerfile multi-stage</code> untuk masing-masing service (base akhir <code>alpine</code> atau <code>scratch</code>). Satu <code>docker-compose.yml</code> di root menjalankan 2 service + 2 database dengan satu perintah.	M3
Kebersihan kode	<code>go fmt ./...</code> tidak menghasilkan perubahan dan <code>go vet ./...</code> bersih.	M1

4. Spesifikasi Endpoint

Kontrak API yang harus dipenuhi. Nama field boleh disesuaikan selama konsisten dan didokumentasikan.

4.1 Course Service (:8081)

Endpoint	Perilaku
POST /courses	Membuat course baru. Field: <code>code</code> (unik), <code>name</code> , <code>description</code> , <code>lecturer</code> , <code>capacity</code> (> 0). Kode yang sudah dipakai ditolak.
GET /courses	Daftar course. Wajib mendukung <i>query param</i> <code>?available=true</code> untuk menyaring yang masih bersisa kuota.
GET /courses/{id}	Detail satu course.

PUT /courses/{id}	Mengubah data course. Kapasitas tidak boleh diturunkan di bawah jumlah kursi yang sudah terpakai.
GET /courses/{id}/availability	Mengembalikan <code>capacity</code> , <code>taken</code> , dan <code>remaining</code> .
POST /courses/{id}/seats/reserve	Endpoint internal. Menambah kursi terpakai sebanyak satu secara atomik. Gagal dengan error domain bila kuota sudah habis.
POST /courses/{id}/seats/release	Endpoint internal. Mengurangi kursi terpakai sebanyak satu. Tidak boleh membuat nilai kursi terpakai menjadi negatif.

Kenapa ada reserve dan release?

Karena pemilik data kuota adalah Course Service. Kalau Enrollment Service yang mengurangi kuota, berarti ada dua pihak yang menulis ke satu data yang sama — dan itulah cara paling cepat membuat microservices menjadi monolit yang terdistribusi. Dua endpoint ini adalah bentuk konkret dari prinsip tersebut, dan akan ditanyakan saat demo.

4.2 Enrollment Service (:8082)

Endpoint	Perilaku
POST /enrollments	Mendaftarkan mahasiswa. Field: <code>student_id</code> , <code>course_id</code> . Alur: validasi input → tanya/booking kursi ke Course Service → simpan enrollment berstatus <code>ACTIVE</code> .
GET /enrollments?student_id=...	Daftar pendaftaran milik satu mahasiswa. Wajib mendukung penyaringan <code>?status=ACTIVE</code> .
GET /enrollments/{id}	Detail satu pendaftaran. Response wajib menyertakan nama course — yang berarti Enrollment Service harus mengambilnya dari Course Service saat dibutuhkan.
DELETE /enrollments/{id}	Membatalkan pendaftaran: status menjadi <code>CANCELLED</code> dan kursi dikembalikan ke Course Service. Membatalkan pendaftaran yang sudah <code>CANCELLED</code> tidak boleh mengembalikan kursi untuk kedua kalinya.

5. Aturan Bisnis Wajib

Inti dari tugas ini. Setiap aturan di bawah akan diuji langsung saat demo.

Kode	Aturan	Yang sedang diuji
BR-01	Jumlah pendaftar tidak boleh melebihi kapasitas, termasuk ketika dua request masuk pada saat yang hampir bersamaan untuk sisa kursi terakhir.	Apakah kamu menjamin ini di level database dalam satu operasi atomik, atau masih memakai pola "SELECT dulu, cek di Go, baru UPDATE" yang bocor saat bersamaan.
BR-02	Satu mahasiswa tidak boleh punya dua pendaftaran <code>ACTIVE</code> pada course yang sama. Percobaan kedua ditolak dengan <code>409</code> .	Apakah aturan ini ditegakkan oleh <i>constraint</i> database, atau hanya oleh pengecekan di kode yang bisa dilewati saat bersamaan.

BR-03	Pembatalan mengembalikan tepat satu kursi. Memanggil <code>DELETE</code> dua kali pada pendaftaran yang sama tidak boleh mengembalikan dua kursi.	Pemahaman idempotensi — mengapa operasi yang sama diulang harus menghasilkan keadaan yang sama.
BR-04	Mendaftar ke <code>course_id</code> yang tidak ada harus dibalas <code>404</code> dengan pesan yang jelas, bukan <code>500</code> .	Apakah error dari service lain diterjemahkan menjadi error domain, atau diteruskan mentah.
BR-05	Bila Course Service mati atau tidak merespons, <code>POST /enrollments</code> wajib membalas <code>503</code> dalam waktu maksimal 3 detik. Tidak boleh menggantung, tidak boleh <i>crash</i> , dan kejadian ini wajib tercatat di log.	Penguasaan <code>context.WithTimeout</code> dan pemahaman bahwa kegagalan service tetangga adalah kondisi normal, bukan kejadian luar biasa.
BR-06	Bila <i>reserve</i> ke Course Service sudah berhasil tetapi penyimpanan enrollment gagal, kursi tidak boleh hilang: sistem wajib memanggil <i>release</i> sebagai kompensasi, atau menuliskan kegagalan itu ke log dengan penanda khusus agar bisa ditelusuri.	Kesadaran bahwa dua database berbeda tidak bisa berbagi satu transaksi — pintu masuk ke konsep <i>compensating transaction</i> .

Catatan tentang BR-01

Ini adalah aturan yang paling sering dianggap sudah beres padahal belum. Cara membuktikannya saat demo: buat course berkapasitas 1, lalu kirim dua request pendaftaran hampir bersamaan. Kalau keduanya berhasil, BR-01 gagal — sekalipun pengujian satu per satu terlihat benar.

6. Kontrak Response & Error

Konsistensi format adalah bagian dari penilaian, bukan hiasan.

6.1 Format amplop response

```
// Sukses
{ "success": true, "data": { ... } }

// Gagal
{ "success": false, "error": { "code": "ERR_NO_SEAT", "message": "kuota course sudah penuh" } }
```

6.2 Pemetaan error

Setiap sentinel error di layer service wajib dipetakan ke satu kode dan satu status HTTP di layer handler. Pemetaan ini dilakukan **di satu tempat saja** — bukan disebar dengan `if` di setiap handler.

Kode	HTTP	Dipakai saat
ERR_VALIDATION	400	Body/parameter tidak lolos validasi
ERR_COURSE_NOT_FOUND	404	Course tidak ada (BR-04)
ERR_ENROLLMENT_NOT_FOUND	404	Enrollment tidak ada
ERR_ALREADY_ENROLLED	409	Pendaftaran ganda (BR-02)
ERR_NO_SEAT	409	Kuota habis (BR-01)
ERR_UPSTREAM_UNAVAILABLE	503	Course Service mati atau timeout (BR-05)

ERR_INTERNAL

500

Sisanya. Detail teknis masuk ke log, **bukan** ke response.**Yang diperiksa penguji**

Response `500` yang muncul untuk kondisi yang seharusnya punya kode sendiri dihitung sebagai kegagalan aturan bisnis terkait. Begitu juga pesan error yang membocorkan isi query SQL atau *stack trace* ke client.

7. Tingkat Pencapaian

Kerjakan berurutan. Tingkat 1 yang rapi jauh lebih bernilai daripada Tingkat 3 yang setengah jadi.

Tingkat	Cakupan	Nilai maks. Implementasi
Tingkat 1 — Wajib Batas kelulusan tugas	Kedua service jalan lewat satu <code>docker compose up</code> . CRUD course berfungsi. Pendaftaran alur normal berhasil. BR-01 dan BR-04 terpenuhi. Format response konsisten.	70
Tingkat 2 — Baik	Seluruh Tingkat 1, ditambah: BR-02, BR-03, dan BR-05 terpenuhi; pembatalan mengembalikan kuota; middleware logging dan recovery aktif; minimal 3 unit test <i>table-driven</i> lulus.	88
Tingkat 3 — Mahir	Seluruh Tingkat 2, ditambah: BR-06 terpenuhi; <code>GET /enrollments/{id}</code> menyertakan data course hasil pemanggilan antar-service; <code>Dockerfile</code> multi-stage menghasilkan image ramping; README menjelaskan keputusan desain secara utuh.	100

Nilai tambahan (opsional, dikerjakan hanya bila Tingkat 3 sudah tuntas)

- Endpoint `GET /enrollments/summary?student_id=...` yang mengambil detail beberapa course secara **paralel** memakai goroutine + `sync.WaitGroup`, dengan satu context timeout bersama.
- Health check* `GET /health` di kedua service yang juga memeriksa koneksi database, lalu dipakai sebagai `healthcheck` di `docker-compose.yml`.
- Mekanisme percobaan ulang (*retry*) sekali dengan jeda untuk pemanggilan *release* yang gagal.

8. Skenario Uji Penerimaan

Inilah urutan yang akan dijalankan di depanmu saat demo. Latih sampai lancar.

Demo **wajib** dimulai dari keadaan bersih:

```
docker compose down -v
docker compose up --build
```

Bila sistem hanya berjalan setelah ada langkah manual di luar perintah di atas, sistem dianggap belum selesai.

No	Langkah	Hasil yang diharapkan
1	Jalankan <code>docker compose up</code> dari keadaan bersih	4 container hidup; kedua service siap menerima request tanpa restart manual
2	Buat course IF-101 dengan <code>capacity: 2</code>	<code>201</code> , data tersimpan
3	<code>GET /courses/{id}/availability</code>	<code>capacity 2, taken 0, remaining 2</code>

4	Daftarkan mahasiswa A	201 ; <code>remaining</code> menjadi 1
5	Daftarkan mahasiswa A lagi ke course yang sama	409 <code>ERR_ALREADY_ENROLLED</code> ; <code>remaining</code> tetap 1 (BR-02)
6	Daftarkan mahasiswa B	201 ; <code>remaining</code> menjadi 0
7	Daftarkan mahasiswa C	409 <code>ERR_NO_SEAT</code> (BR-01)
8	Batalkan pendaftaran mahasiswa A	200 ; status <code>CANCELLED</code> ; <code>remaining</code> kembali menjadi 1 (BR-03)
9	Batalkan pendaftaran mahasiswa A sekali lagi	Ditolak; <code>remaining</code> tetap 1 (BR-03)
10	Daftar ke <code>course_id</code> yang tidak ada	404 <code>ERR_COURSE_NOT_FOUND</code> (BR-04)
11	<code>docker compose stop course-service</code> , lalu coba mendaftar	503 <code>ERR_UPSTREAM_UNAVAILABLE</code> dalam ≤ 3 detik; Enrollment Service tetap hidup; log mencatat kejadian (BR-05)
12	<code>docker compose start course-service</code> , ulangi pendaftaran	Kembali normal tanpa perlu me-restart Enrollment Service
13	Buat course kapasitas 1, kirim dua pendaftaran hampir bersamaan	Tepat satu berhasil, satu ditolak 409 ; <code>taken</code> tidak pernah melebihi 1 (BR-01)
14	<code>go test ./...</code> di Enrollment Service	Seluruh test lulus

9. Deliverable

Yang dikumpulkan dan bagaimana bentuknya.

9.1 Struktur repository

Satu repository berisi kedua service, dengan pemisahan folder yang jelas:

```

quickcourse/
├── docker-compose.yml
├── README.md
├── course-service/
│   ├── cmd/ atau main.go
│   ├── internal/
│   │   ├── handler/
│   │   ├── service/
│   │   ├── repository/
│   │   └── domain/          # struct + sentinel error
│   ├── db/init.sql
│   ├── Dockerfile
│   └── go.mod
├── enrollment-service/
│   └── ... struktur serupa, termasuk client/ untuk pemanggilan Course Service
└── postman/QuickCourse.postman_collection.json

```

9.2 Isi README

1. Cara menjalankan — cukup satu perintah, plus daftar port yang dipakai.
2. Diagram/penjelasan arsitektur singkat beserta alasan pembagian domain.

3. Daftar endpoint dengan contoh request dan response.
4. **Bagian "Keputusan Desain"**: minimal empat keputusan yang kamu ambil beserta alasannya (misalnya cara menjamin BR-01, alasan pemilihan status HTTP, cara menangani kegagalan Course Service). Bagian ini dibaca penguji sebelum demo dimulai.
5. Bagian "Keterbatasan": apa yang belum sempat dikerjakan dan apa risikonya. Menuliskan keterbatasan dengan jujur bernilai lebih tinggi daripada menyembunyikannya.

9.3 Pengumpulan

- Kirim tautan repository GitHub (publik atau kolaborator ditambahkan) sebelum **pukul 15.30** pada hari pertemuan Minggu 4.
- Pukul 15.30 adalah *code freeze*. Commit setelah jam itu tidak dinilai — sisa waktu dipakai untuk merapikan README dan menyiapkan demo.
- Riwayat commit ikut dilihat: satu commit berisi seluruh proyek menimbulkan pertanyaan tambahan saat sesi pertahanan.

10. Aturan Main

Batasan yang membatalkan penilaian, dan sikap terhadap bantuan luar.

Yang membatalkan aspek terkait

- Enrollment Service mengakses database Course Service secara langsung.
- Kedua service berbagi satu database yang sama.
- Kuota disimpan atau diubah di sisi Enrollment Service.
- *Connection string*, kredensial, atau URL service lain ditulis *hardcode* di dalam kode.
- Sistem hanya bisa berjalan lewat `go run` di laptop sendiri, tidak lewat Docker Compose.

Tentang bantuan AI, tutorial, dan potongan kode dari internet

Semuanya boleh dipakai, tanpa pengecualian — itu bagian normal dari pekerjaan seorang backend developer. Syaratnya satu: **kamu harus bisa menjelaskan setiap baris yang ada di repositorimu**. Saat sesi pertahanan, penguji akan menunjuk beberapa bagian kode secara acak dan memintamu menjelaskan fungsinya serta apa yang terjadi bila baris itu dihapus. Bagian yang tidak bisa dijelaskan dianggap tidak dikerjakan, sekalipun berjalan sempurna. Karena itu, menyalin kode yang belum kamu pahami justru merugikan.

11. Pertanyaan Pertahanan

Empat pertanyaan akan dipilih acak dari daftar ini saat demo. Tidak ada pertanyaan di luar daftar.

Fondasi Go **MINGGU 1**

1. Kenapa repository dideklarasikan sebagai interface? Apa yang menjadi sulit kalau service langsung memakai tipe konkretnya?
2. Apa beda `errors.Is` dengan perbandingan `==` biasa, dan kenapa pembungkusan `%w` diperlukan?
3. Tunjukkan satu method yang kamu buat dengan *pointer receiver* dan jelaskan kenapa bukan *value receiver*.
4. Apa yang sebenarnya diuji oleh unit test-mu, dan kondisi apa yang **tidak** bisa ditangkap oleh test itu?

REST API & database **MINGGU 2**

5. Tunjukkan baris kode yang menjamin kuota tidak bisa minus saat dua request masuk bersamaan. Kenapa cara itu aman?

- Kenapa kuota habis dijawab `409` dan bukan `400` ? Apa bedanya makna dua status itu?
- Kalau seluruh isi file service dipindahkan ke handler, apa yang secara nyata menjadi lebih buruk? Sebutkan yang konkret, bukan "kurang rapi".
- Di mana pemetaan dari error domain ke status HTTP terjadi, dan kenapa di situ?

Microservices & Docker MINGGU 3

- Kenapa alamat Course Service ditulis `http://course-service:8081` dan bukan `localhost:8081` ?
- `depends_on` sudah dipasang, kenapa aplikasi masih bisa gagal terhubung ke database saat `up` pertama kali? Bagaimana kamu mengatasinya?
- Kalau `context.WithTimeout` dihapus, apa persisnya yang terjadi ketika Course Service hidup tapi tidak pernah membalas?
- Reserve ke Course Service sudah berhasil, lalu penyimpanan enrollment gagal. Apa yang terjadi pada kuota, dan bagaimana kamu menanganinya?
- Kenapa memakai *multi-stage build*? Berapa besar image-mu, dan berapa kira-kira kalau tanpa itu?
- Besok ada Payment Service yang harus dipanggil sebelum pendaftaran dicatat. Bagian mana saja yang berubah, dan bagian mana yang tidak perlu disentuh sama sekali?
- Menurutmu, untuk kasus sekecil QuickCourse, apakah pemisahan dua service ini sepadan? Jawaban "tidak" diterima selama alasannya kuat.

12. Rubrik Penilaian

Selaras dengan skema evaluasi program.

Aspek	Bobot	Indikator
Pemahaman Konsep	25%	Jawaban pada sesi pertahanan (Bagian 11) tepat, memakai bahasa sendiri, dan menunjukkan alasan di balik pilihan — bukan hafalan istilah.
Implementasi Teknis	40%	Tingkat pencapaian (Bagian 7) dan jumlah skenario uji penerimaan (Bagian 8) yang lulus, serta kesesuaian dengan ketentuan teknis wajib (Bagian 3).
Progres & Konsistensi	20%	Catatan progres Minggu 1–3, keaktifan saat sesi eksplorasi, dan riwayat commit yang menunjukkan pengerjaan bertahap.
Presentasi Akhir	15%	Demo berjalan tanpa kepanikan, penjelasan runtut dalam batas waktu, README terbaca, dan kejujuran menyampaikan keterbatasan.

Format demo (5–10 menit)

- 30 detik** — sebutkan pembagian service dan alasannya.
- 1 menit** — `docker compose down -v` & `docker compose up` di depan penguji.
- 3–4 menit** — jalankan skenario uji penerimaan Bagian 8, terutama nomor 5, 7, 9, 11, dan 13.
- 1 menit** — tunjukkan satu bagian kode yang paling kamu banggakan dan jelaskan kenapa.
- 2–3 menit** — sesi pertahanan.

Lampiran A — Contoh Skema & Payload

Titik awal. Silakan dikembangkan selama kontrak di Bagian 4–6 tetap terpenuhi.

db_course — init.sql

```
CREATE TABLE courses (  
  id          SERIAL PRIMARY KEY,  
  code        VARCHAR(20) NOT NULL UNIQUE,  
  name        VARCHAR(150) NOT NULL,  
  description TEXT,  
  lecturer    VARCHAR(100) NOT NULL,  
  capacity    INT NOT NULL CHECK (capacity >= 0),  
  taken       INT NOT NULL DEFAULT 0 CHECK (taken >= 0),  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),  
  CONSTRAINT taken_tidak_melebihi_kapasitas CHECK (taken <= capacity)  
);
```

db_enrollment — init.sql

```
CREATE TABLE enrollments (  
  id          SERIAL PRIMARY KEY,  
  student_id  VARCHAR(30) NOT NULL,  
  course_id   INT          NOT NULL,  -- tanpa foreign key: tabel course ada di database lain  
  status      VARCHAR(15) NOT NULL DEFAULT 'ACTIVE',  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
  
-- Penegakan BR-02 di level database  
CREATE UNIQUE INDEX satu_pendaftaran_aktif  
  ON enrollments (student_id, course_id)  
  WHERE status = 'ACTIVE';
```

Contoh request & response

```
POST /enrollments  
{ "student_id": "241511001", "course_id": 1 }  
  
201 Created  
{ "success": true,  
  "data": { "id": 7, "student_id": "241511001", "course_id": 1, "status": "ACTIVE" } }  
  
409 Conflict  
{ "success": false,  
  "error": { "code": "ERR_NO_SEAT", "message": "kuota course IF-101 sudah penuh" } }
```

Lampiran B — Checklist Mandiri Sebelum Demo

Periksa satu per satu sebelum jam 15.30.

✓ Butir

- ☐ Repository di-clone ke folder baru, lalu `docker compose up` — sistem berjalan tanpa langkah manual tambahan
- ☐ Tidak ada `localhost` yang tersisa di konfigurasi antar-container

- ☐ File `.env` berisi kredensial asli tidak ikut ter-commit; tersedia `.env.example`
- ☐ Seluruh 14 langkah pada Bagian 8 sudah dicoba sendiri, bukan diasumsikan berhasil
- ☐ `go fmt ./...` tidak mengubah apa pun; `go vet ./...` bersih; `go test ./...` lulus
- ☐ Tidak ada endpoint yang membalas `500` untuk kondisi yang sudah punya kode error sendiri
- ☐ README memuat bagian "Keputusan Desain" dan "Keterbatasan"
- ☐ Postman collection tersimpan di repository dan sudah diurutkan sesuai alur demo
- ☐ Setiap anggota kelompok sudah mencoba menjawab seluruh pertanyaan Bagian 11 dengan suara keras

Penutup

QuickCourse hanya dua service dan tujuh tabel kolom. Sistem sungguhan punya puluhan. Yang membedakan bukan jumlahnya, melainkan apakah kamu terbiasa memikirkan kegagalan sejak awal — kuota yang direbut bersamaan, service tetangga yang mati, operasi yang diulang dua kali. Kalau tiga hal itu sudah menjadi pertanyaan refleksi saat kamu menulis kode, pelatihan ini berhasil.